МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного
бюджетного образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА» Техникум Пермского института (филиала)

Практическая работа №1

по дисциплине: Поддержка и тестирование программных модулей

Руководитель:
Д.Б.Берестов /И.О.Фамилия/
“__30__”_октября_2026
Исполнитель:
А.В.Язев /И.О.Фамилия/
“__30__”октября__2026

Пермь,2026

Оглавление

1. Цель работы 2
2. Описание проекта 2
3. Целевая платформа 2
4. Ход выполнения работы 3
5. Полученные результаты 5
6. Выводы 5

Отчёт по практической работе

«Модульное тестирование в Visual Studio (MSTest)»

1. Цель работы

Освоить методику создания и выполнения модульных тестов для управляемого кода на платформе .NET с использованием MSTest в среде Visual Studio. Научиться выявлять и исправлять ошибки с помощью тестов, а также применять подход разработки на основе тестирования (TDD).

2. Описание проекта

Был разработан проект Bank, содержащий класс BankAccount, моделирующий банковский счёт. Класс предоставляет методы для пополнения (Debit) и получения баланса. Исходно в методе Debit была допущена логическая ошибка: вместо вычитания суммы выполнялось сложение. Для проверки корректности работы создан тестовый проект BankTests с использованием фреймворка MSTest.

3. Целевая платформа

Оба проекта ориентированы на .NET 10.0. Используется мета-пакет MSTest версии 4.0.1.

4. Ход выполнения работы

Написание первого теста

Был создан метод Debit_WithValidAmount_UpdatesBalance, который проверяет,

что после списания допустимой суммы баланс уменьшается на заданную величину. При первом запуске тест завершился ошибкой, так как в коде использовалось сложение вместо вычитания.

```
[TestMethod]
public void Debit_WithValidAmount_UpdatesBalance()
{
double beginningBalance = 11.99;
double debitAmount = 4.55;
double expected = 7.44;
var account = new BankAccount("Mr. Bryan Walton", beginningBalance);
account.Debit(debitAmount);
Assert.AreEqual(expected, account.Balance, 0.001);
}
```

Исправление кода

В методе Debit класса BankAccount строка m_balance += amount; была заменена на

m_balance -= amount;. После этого тест стал проходить успешно.

Добавление тестов для граничных условий

Были добавлены два теста, проверяющие выброс исключений при некорректных аргументах:

Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange

Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange

Из-за непредвиденной ошибки (метод Assert.ThrowsException не распознавался), тесты были реализованы через конструкцию try/catch с проверкой сообщения исключения.

```
[TestMethod]
public  void
Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
{
double beginningBalance = 11.99;
double debitAmount = -100.00;
BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);
try
{
account.Debit(debitAmount);
Assert.Fail("Исключение не было выброшено");
}
catch (ArgumentOutOfRangeException e)
{
StringAssert.Contains(e.Message,
BankAccount.DebitAmountLessThanZeroMessage);
}
}
```

Рефакторинг кода и тестов

В класс BankAccount были добавлены публичные константы для сообщений

об ошибках:

```
public const string DebitAmountExceedsBalanceMessage = "Debit amount
exceeds
balance";
public const string DebitAmountLessThanZeroMessage = "Debit amount is less
than
zero";
```

Конструктор исключения в методе Debit был заменён на версию с

сообщением:

throw new ArgumentOutOfRangeException(nameof(amount), amount, DebitAmountLessThanZeroMessage);

Тесты были дополнены проверкой корректности сообщения, что повысило надёжность проверок.

Финальный запуск тестов

После всех исправлений все три теста успешно проходят. Ниже представлен скриншот обозревателя тестов в Visual Studio:

5. Полученные результаты

Разработан класс BankAccount с корректной логикой списания средств.

Создан набор из трёх модульных тестов, покрывающих:

основной сценарий (успешное списание);

попытку списания отрицательной суммы;

попытку списания суммы, превышающей баланс.

Все тесты выполняются успешно, что подтверждает правильность реализации метода Debit.

6. Выводы

В ходе работы были освоены:

создание тестовых проектов в Visual Studio;

написание и запуск модульных тестов с помощью MSTest;

использование утверждений (Assert, StringAssert);

обработка и проверка исключений в тестах;

подход «красный-зелёный-рефакторинг» (TDD).

Модульное тестирование позволило своевременно выявить и исправить ошибку, а также обеспечить защиту от регрессий при дальнейших изменениях кода.